STATIC APPLICATION SECURITY TESTING

ReNew Solar Marketing Website

Source Code Security Analysis Report

CONFIDENTIAL. This document contains source-level findings, file paths and code excerpts. Distribute only to personnel responsible for maintaining the codebase described. Findings reflect the source tree at the commit stated below.

CODEBASE

renew — ReNew Solar Marketing Site

REPORT DATE

17 August 2026

COMMIT ANALYSED

`a4f4020 (main)`

ANALYSIS TYPE

White-box — full source access

LANGUAGES

TypeScript, TSX, CSS

SCALE

6,689 lines · 41 files

FRAMEWORK

Next.js 16.2.10 · React 19.2.4

REPORT VERSION

1.1 — Post-remediation

REMEDIATION APPLIED

17 August 2026

STATUS

8 of 9 closed · 1 mitigated

1 Executive Summary

A static application security test was performed against the complete ReNew Solar marketing website source tree — 6,689 lines across 41 files, comprising all application code, build scripts and configuration. The analysis combined three independent automated engines with a line-by-line manual review of every file that handles input, renders dynamic content, or performs calculation.

AUTOMATED ANALYSIS RESULT

All three automated engines returned zero findings. Semgrep executed 129 security rules across 7 rulesets against all 40 files with 100% parse coverage. ESLint, running the Next.js core-web-vitals and TypeScript configurations, reported no errors or warnings. The TypeScript compiler passed under `strict: true` with no diagnostics. The codebase contains no `any` types, no `@ts-ignore` directives and no `eslint-disable` comments — nothing has been suppressed to achieve those results.

That is a genuinely strong automated baseline, and it means the substance of this report comes from manual review. Automated SAST is effective at pattern-matching known-dangerous constructs; it cannot evaluate whether a calculation is correct, whether a validation control is actually enforced, or whether a form does what its surrounding copy promises. Each of the three most significant findings below falls into that category, and none were detected by any of the automated tools.

No injection, code-execution, or memory-safety vulnerability was found. The codebase contains no `eval`, no `new Function`, no `innerHTML` assignment and no `document.write`. It reads no cookies, uses no `localStorage` or `sessionStorage`, registers no `postMessage` handler, and performs no network requests of any kind.

Findings by severity — as reviewed (v1.0)

0	1	2	1	5
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

POST-REMEDIATION STATUS (V1.1)

8 of 9 findings closed, the three behavioural ones confirmed by runtime testing. The remaining item (SAST-03, the submission endpoint) is mitigated but open by client decision. Dependencies went from 12 advisories to **zero**, and all three static engines still report 0 findings across a now-larger file set. Detail in Section 1A.

The headline result of the manual review

The single most security-relevant code path in the application — the only place where attacker-influenceable data enters — is **correctly and deliberately secured**. Two components read `window.location.hash` and validate it against a closed `Set` of known identifiers before use. This is textbook allow-list sanitisation, and it is documented as a positive finding in Section 5.

The defects that were found are concentrated instead in **correctness and enforcement**: a savings calculator that ignores its own tariff data and reports financial projections in error by up to 54%; a required-field control that is

not actually enforced; and three forms that collect personal data and discard it. These are the kinds of defect that a scanner is structurally incapable of finding, and they carry more real-world consequence for this application than any of the pattern-level issues a scanner would have flagged.

> **CROSS-REFERENCE**
>
> Findings **SAST-03**, **SAST-05** and the dependency position correspond to VAPT-02, VAPT-07 and VAPT-04 in the VAPT report of the same date, reached independently from the source side. Severities are kept consistent between the two documents. Remediate each once.

1A Remediation Status

Remediation was carried out on the same day version 1.0 was delivered. Every code-level finding was fixed. The findings in Section 5 are left as written — they describe the code as reviewed — and this section records what changed and how it was confirmed.

ID	SEVERITY	FINDING	STATUS	VERIFICATION
SAST-03	HIGH	Form submissions discarded	MITIGATED	Misleading promise removed; **endpoint deferred by client**
SAST-01	MEDIUM	Calculator ignores its tariff table	CLOSED	**Runtime-verified** — four states now yield four distinct correct figures
SAST-02	MEDIUM	Required constraint not enforced	CLOSED	**Runtime-verified** — `checkValidity()` now returns `false` when empty
SAST-04	LOW	Numeric bounds unenforced	CLOSED	**Runtime-verified** — clamped at 100,000 with a visible notice
SAST-05	INFO	JSON-LD sink unescaped	CLOSED	Escape applied; built HTML inspected
SAST-06	INFO	Build scripts excluded from type checking	CLOSED	`tsc --noEmit` passes with `scripts/` included
SAST-07	INFO	Non-null assertion	CLOSED	Replaced with a conditional; zero assertions remain
SAST-08	INFO	Unsound type assertions in hash validators	CLOSED	Type predicates; **11-payload regression test still passes**
SAST-09	INFO	IDE config committed; dev IP in config	CLOSED	`.idea/` untracked and ignored; IP moved to env var

1A.1 Post-remediation tool run

All engines re-run against the remediated tree. The clean baseline is preserved:

ENGINE	BEFORE	AFTER	COVERAGE CHANGE
Semgrep (129 rules)	0	0	40 → 43 files
ESLint	0	0	40 → 43 files
TypeScript `strict`	0	0	now includes `scripts/`
`bun audit`	12	0	all five packages patched
`next build`	PASS	PASS	7 → 9 static routes

1A.2 Key code changes

```
// src/lib/solar.ts — bounds clamped in the domain layer, so any future
// server-side caller inherits the guarantee (SAST-04)
export const MAX_MONTHLY_UNITS = 100_000;
const monthlyUnits = Math.min(MAX_MONTHLY_UNITS, Math.max(0, Number(usage) || 0));

// savings-calculator.tsx — the tariff table is now the single source of unit
// cost; the flat DEFAULT_UNIT_COST that shadowed it is gone (SAST-01)
const tariffFor = (state: string) => (STATE_TARIFFS[state] ?? 8).toFixed(2);

// solar-module-detail.tsx — guard is now load-bearing: remove it and the code
// stops compiling, rather than silently narrowing an arbitrary string (SAST-08)
function isModuleId(value: string): value is ModuleRange["id"] {
  return (moduleIds as ReadonlySet<string>).has(value);
}
```

A CORRECTION MADE DURING REMEDIATION

The fix recommended in SAST-02 needed adjusting when applied. The report proposed a visually-hidden input carrying `required`; the first implementation also set `readOnly`, which **bars an element from constraint validation entirely** and would have left the finding unfixed while appearing addressed. The shipped version omits `readOnly`, uses a no-op `onChange`, and keeps the element laid out and click-through rather than `display:none` — Chrome refuses to submit and logs "not focusable" when an invalid control is fully hidden. Runtime testing confirmed both that submission is now blocked and that no console error is produced.

REMAINING OPEN ITEM

SAST-03 is mitigated, not closed. The submission endpoint was deferred by the client, so the forms still transmit nothing — the change removes the misleading commitment and routes users to the phone and email that do reach the team. The `TODO(batch-4-followup)` markers remain in the source as the tracking record. When the endpoint is built, SAST-02's client-side control must not be treated as validation: the server has to validate independently.

Separately, the **Customer category** control noted under SAST-01 remains non-functional by client decision. The state tariff defect — the part causing materially wrong figures — is fixed; the category selector still does not affect the calculation. The disclaimer was reworded to say results are an average *residential* tariff and that rates vary by customer category, so the UI no longer overstates its own precision.

2 Scope, Tooling & Coverage

2.1 Code analysed

AREA	FILES	LINES	COVERAGE
Application source `src/` (TS/TSX)	35	6,010	100%
Stylesheet `src/app/globals.css`	1	564	100%
Build scripts `scripts/`	2	76	100%
Configuration (next / eslint / postcss)	3	39	100%
Total	**41**	**6,689**	COMPLETE

Of the 35 TypeScript source files, 17 are client components carrying the `"use client"` directive — these received priority review, as they are the only code that executes in the browser with access to the DOM and user input. Third-party code under `node_modules/` was excluded from static analysis and assessed separately by composition analysis (Section 6).

2.2 Automated engines

ENGINE	VERSION	CONFIGURATION	FINDINGS	COVERAGE
Semgrep	`1.173.0`	7 rulesets, 129 rules	0	40 files, ~100% lines parsed
ESLint	`9.x (flat)`	`core-web-vitals` + `typescript`	0	40 files
TypeScript	`5.x`	`strict: true`, `noEmit`	0	All included files
Manual review	—	Line-by-line, taint & logic	9	41 files

Semgrep rulesets applied:

```
┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
 • Findings: 0 (0 blocking)
 • Rules run: 129
 • Targets scanned: 40
 • Parsed lines: ~100.0%
```

WHY THE MANUAL PASS MATTERS HERE

A zero-finding automated result is easy to misread as "the code is secure". What it accurately says is that the code contains none of the *recognisable dangerous patterns* these 129 rules encode. It says nothing about whether the code is *correct*. Every finding in Section 5 was found by reading the code and reasoning about intent — including one where the correct data was present in the file and simply never used.

2.3 Manual review methodology

1. **Sink inventory.** Enumerate every dangerous sink in the codebase — raw HTML injection, dynamic evaluation, navigation, storage, and cross-document messaging.
2. **Source inventory.** Enumerate every point where data outside the developer's control enters — URL, user input, external data.
3. **Taint tracing.** For every source, trace whether it can reach any sink, and evaluate the sanitisation applied on each path.
4. **Control enforcement review.** Verify that each stated validation control (required fields, min/max bounds, type constraints) is actually enforced, rather than merely declared.
5. **Business-logic review.** Verify that calculations produce correct results and that user-facing claims match implemented behaviour.
6. **Resource lifecycle review.** Verify that every event listener, observer and timer is cleaned up.
7. **Configuration & hygiene review.** Type-checking coverage, committed artifacts, and internal detail exposed in the repository.

2.4 Standards applied

CWE for weakness classification, **OWASP Top 10:2021** for risk categorisation, and **OWASP ASVS v4.0** for verification requirements. Severity uses the same environmentally-adjusted framework as the companion VAPT report: findings are rated for this deployment — a static export with no server-side runtime — rather than for a theoretical worst case.

3 Taint Analysis

The core of any SAST engagement is establishing which untrusted data can reach which dangerous operation. Both sides of that question were enumerated exhaustively.

3.1 Sink inventory

DANGEROUS SINK	OCCURRENCES	RESULT
`eval()`	0	Absent
`new Function()`	0	Absent
`innerHTML`	0	Absent
`document.write()`	0	Absent
`setTimeout/setInterval`	0	No string-argument form; all take function references
`dangerouslySetInnerHTML`	1	Reviewed — static data only, see SAST-05
`window.location`	2	Both are **reads** of `.hash`; no assignment, no redirect sink
`window.open`	0	Absent
`localStorage / sessionStorage`	0	Absent — no client-side persistence at all
`document.cookie`	0	Absent
`postMessage`	0	Absent — no cross-document messaging
`fetch / XHR / axios`	0	Absent — the application makes no network requests

3.2 Source inventory

UNTRUSTED INPUT SOURCE	PRESENT	NOTES
URL fragment (`location.hash`)	YES	The only attacker-influenceable source — analysed in 3.3
Form input fields	YES	7 fields across 3 forms; never leave the browser (SAST-03)
URL query parameters	NO	Never read — no `useSearchParams` or equivalent
Route parameters	NO	No dynamic route segments exist
HTTP request data	NO	No server-side code
Cookies / storage	NO	Not used
External API responses	NO	No network calls
`postMessage` events	NO	No listener registered

3.3 The one real taint path — and why it is safe

Two components deep-link to a specific tab via the URL fragment. This is the only place in 6,689 lines where a value an attacker can control enters the application. Both implement the same pattern:

```
// src/components/sections/solar-module-detail.tsx:12
const moduleIds = new Set<ModuleRange["id"]>(
  moduleRanges.map((moduleRange) => moduleRange.id),
);

// :16 — SOURCE: attacker-controlled URL fragment
function getModuleIdFromHash(): ModuleRange["id"] | null {
  const hash = window.location.hash.replace("#", "");
  return moduleIds.has(hash as ModuleRange["id"])   // ← closed-set allow-list
    ? (hash as ModuleRange["id"])
    : null;                                          // ← anything unknown becomes null
}
```

The identical construction appears at `src/components/sections/manufacturing-detail.tsx:18` using a `plantIds` set. The validated value is subsequently passed to `document.getElementById()` for scroll positioning — a sink that would be susceptible to DOM-clobbering if fed arbitrary input.

> **VERIFIED SECURE — NO FINDING**
>
> The `Set.has()` check is an **allow-list against a closed set derived from static data**. A fragment value that is not an exact member returns `null` and the calling component falls back to its default tab. There is no path by which attacker-supplied text reaches the DOM, a URL, or any rendering sink.
>
> This is the correct pattern, and notably it is *allow-listing* rather than the far more common and far weaker approach of filtering known-bad characters. It should be preserved as-is; a future refactor that replaced the `Set` check with a regular expression or a string comparison would be a security regression.

3.4 Dynamic attribute construction

49 JSX attributes build `href` or `src` values from variables rather than string literals — normally a priority area, since a variable-driven `href` is the classic vector for `javascript:` URI injection and open redirect. Every one was traced to its origin.

All 49 resolve to hard-coded local data — module-scope constant arrays and objects declared in the same file, containing relative paths such as `/downloads/certificates/ul-61730-certificate.pdf` or absolute `https://` URLs to known ReNew properties. None derives from user input, the URL, or any external source. There is no open-redirect and no `javascript:` injection vector anywhere in the codebase.

3.5 Resource lifecycle

Every registered listener and timer was checked for a matching teardown, since an unreleased listener on a long-lived page is both a memory leak and a potential source of stale-closure logic errors.

FILE	REGISTERED	RELEASED	RESULT
`contact-modal.tsx`	2	2	BALANCED
`custom-dropdown.tsx`	1	1	BALANCED
`project-showcase.tsx`	3	3	BALANCED
`manufacturing-detail.tsx`	2	2	BALANCED
`solar-module-detail.tsx`	1	1	BALANCED
`who-we-serve.tsx`	2	2	BALANCED
`channel-partners.tsx`	2	2	BALANCED
Timers (4 files)	5	6	ALL CLEARED

No leaks found. The modal's focus trap (`contact-modal.tsx:136–176`) was reviewed in detail and correctly restores `body` overflow, removes its keydown handler, and returns focus to the opening element on teardown.

4 Findings Summary

ID	SEVERITY	FINDING	CWE	LOCATION
SAST-03	HIGH	Form submissions discarded; success state unconditional	CWE-670	3 components
SAST-01	MEDIUM	Savings calculator ignores its own state tariff table	CWE-682	`savings-calculator.tsx`
SAST-02	MEDIUM	Required-field constraint declared but not enforced	CWE-1173	`custom-dropdown.tsx:100`
SAST-04	LOW	Numeric bounds not enforced; unbounded calculation input	CWE-1284	`savings-calculator.tsx`
SAST-05	INFO	Raw HTML sink for JSON-LD without `<` escaping	CWE-79	`page.tsx:33`
SAST-06	INFO	Build scripts excluded from type checking	CWE-1109	`tsconfig.json`
SAST-07	INFO	Non-null assertion bypasses strict null safety	CWE-476	`footer.tsx:158`
SAST-08	INFO	Unsound type assertions in hash validators	CWE-704	2 components
SAST-09	INFO	IDE configuration committed; internal dev IP in config	CWE-200	`.idea/`, `next.config.ts`

4.1 Verified clean

Explicitly checked and found sound. Recorded so a future change does not silently regress them.

CHECK	RESULT	EVIDENCE
Code injection sinks	CLEAN	No `eval`, `new Function`, string-form timers
DOM XSS sinks	CLEAN	No `innerHTML` or `document.write`; single reviewed React sink
URL fragment handling	SECURE	Closed-set allow-list validation — Section 3.3
Open redirect	CLEAN	All 49 dynamic `href` / `src` trace to static data
Hardcoded secrets	CLEAN	Semgrep `p/secrets` 0 findings; no `process.env` in `src/`
Type-safety escapes	CLEAN	No `any`, `as any`, or `@ts-ignore` anywhere
Lint suppressions	CLEAN	No `eslint-disable` comment in the codebase
Resource leaks	CLEAN	All listeners, observers and timers released — Section 3.5
Client-side storage	CLEAN	No storage API used; no PII persisted in the browser
Cross-document messaging	CLEAN	No `postMessage` sender or listener
Reverse tabnabbing	CLEAN	All 5 `target="_blank"` carry `rel="noopener noreferrer"`
Regular-expression safety	CLEAN	No nested quantifiers; no ReDoS-capable pattern (see 8.2)
Build-script injection	CLEAN	Hardcoded paths; no shell execution or dynamic input (see 8.2)
External CSS resources	CLEAN	Only `@import "tailwindcss"`; no remote `url()`

5 Detailed Findings

SAST-03 Form submissions discarded; success state unconditional

HIGH

CWE	OWASP	DETECTED BY	CROSS-REF
CWE-670 — Always-Incorrect Control Flow	A04:2021 — Insecure Design	Manual review only	VAPT-02

DESCRIPTION

All three data-collection forms implement submission entirely client-side. Each handler suppresses the default browser submit, sets a local boolean, and returns. No transport exists — consistent with the taint analysis in Section 3.1, which found zero `fetch` or `XMLHttpRequest` calls in the entire codebase.

EVIDENCE

```
// src/components/sections/contact.tsx:105
/* TODO(batch-4-followup): external form endpoint — static export has no
   server; submission currently just confirms locally. */
<form onSubmit={(e) => { e.preventDefault(); setSubmitted(true); }}>

// src/components/contact-modal.tsx:284
onSubmit={(event) => { event.preventDefault(); setSubmitted(true); }}

// src/components/sections/savings-calculator.tsx:117
onSubmit={(e) => { e.preventDefault(); setHasSubmitted(true); }}
```

COMPONENT	LINE	PERSONAL DATA CAPTURED AND DISCARDED
`sections/contact.tsx`	105	name, company, phone, state, requirement
`contact-modal.tsx`	284	name, company, phone, **email**, requirement
`sections/savings-calculator.tsx`	117	state, monthly consumption

IMPACT

- Every sales enquiry submitted since launch has been lost. The adjacent copy at `contact.tsx:88` commits to a response "within 24 hours".
- The success state is **unconditional**. Even once an endpoint exists, this control flow will report success on network failure or server error unless restructured — the defect is architectural, not just a missing call.
- Personal data is collected without any transport security decision having been made, because no transport exists yet.

REMEDIATION

Restructure the handler so the success state is *derived from a server response*, never set optimistically:

```
const [status, setStatus] = useState<"idle"|"sending"|"ok"|"error">("idle");

async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
  e.preventDefault();
  setStatus("sending");
  try {
    const res = await fetch("/api/enquiry", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(Object.fromEntries(new FormData(e.currentTarget))),
    });
    setStatus(res.ok ? "ok" : "error");   // success reflects the server, not the click
  } catch {
    setStatus("error");
  }
}
```

The receiving endpoint must implement server-side validation, rate limiting and anti-automation — see Section 6.1 of the companion VAPT report. Until it exists, remove the 24-hour commitment from the UI copy.

SAST-01 Savings calculator ignores its own state tariff table

MEDIUM

CWE	ALSO	DETECTED BY	FILE
CWE-682 — Incorrect Calculation	CWE-1164 — Irrelevant Code	Manual review only	`savings-calculator.tsx`

DESCRIPTION

The component defines `STATE_TARIFFS`, a 36-entry map of Indian states and union territories to their electricity tariff in ₹/kWh. It presents a state selector to the user. **The tariff values are never read.** The map is consumed only for its keys, to populate the dropdown's option list:

```
// src/components/sections/savings-calculator.tsx:9
const STATE_TARIFFS: Record<string, number> = {
  Delhi: 8, Karnataka: 8.2, Kerala: 7.1, Ladakh: 5.2, Maharashtra: 11.0, /* …36 entries */
};
const STATES = Object.keys(STATE_TARIFFS);   // ← only the keys are ever used
const DEFAULT_UNIT_COST = "8.00";

// :76 — selecting a state updates a label, but never the tariff
const changeState = (next: string) => {
  setState(next);
  setLocationStatus("Selected manually");   // no setUnitCost(STATE_TARIFFS[next])
};

// :90 — the calculation only ever sees the flat default, or a manual override
const result = useMemo(() => calculateSolar(usage, unitCost), [usage, unitCost]);
```

Confirmed by exhaustive reference search: `STATE_TARIFFS` appears at exactly two lines — its declaration (9) and `Object.keys()` (47). `setUnitCost` is called only to initialise state and to reset to `DEFAULT_UNIT_COST`. The **Customer category** radio group (residential / commercial / industrial) is likewise never read by the calculation — it affects only its own button styling.

IMPACT

Every user is quoted savings at a flat ₹8.00/kWh regardless of the state they select. The quoted figures are wrong for 33 of the 36 states in the table. Measured error for a representative 500 kWh/month household:

STATE SELECTED	TRUE TARIFF	CORRECT ANNUAL	SHOWN ANNUAL	ERROR
Ladakh	₹5.20	₹31,200	₹48,000	**+53.8% overstated**
Jammu and Kashmir	₹5.50	₹33,000	₹48,000	+45.5% overstated
Kerala	₹7.10	₹42,600	₹48,000	+12.7% overstated
Karnataka	₹8.20	₹49,200	₹48,000	−2.4% understated
Maharashtra (default)	₹11.00	₹66,000	₹48,000	**−27.3% understated**

Over the 30-year lifetime figure the site also displays, the Ladakh case overstates savings by **₹5,04,000**. Note that the page's default state, Maharashtra, is *understated* by 27% — so the defect also costs ReNew a materially stronger sales figure in its own default view.

The page carries a disclaimer that "calculations use an average electricity tariff", which partially covers the flat rate. It does not reconcile with a UI that presents a 36-option state selector implying a state-specific result, while the correct per-state data sits unused in the same file.

REMEDIATION

Wire the table to the calculation — the data is already correct and present:

```
const changeState = (next: string) => {
  setState(next);
  setLocationStatus("Selected manually");
  if (!isManualCost) setUnitCost(STATE_TARIFFS[next].toFixed(2));
};

// Initialise consistently rather than from a hardcoded constant:
const [state, setState] = useState("Maharashtra");
const [unitCost, setUnitCost] = useState(STATE_TARIFFS["Maharashtra"].toFixed(2));

// And restore the state tariff — not a flat default — when leaving manual mode:
const toggleUnitCostMode = () => {
  if (isManualCost) setUnitCost(STATE_TARIFFS[state].toFixed(2));
  setIsManualCost(!isManualCost);
};
```

Separately, either apply `category` to the calculation (commercial and industrial tariffs differ materially from residential in every Indian state) or remove the control. A radio group that changes nothing misleads the user about the precision of the estimate. Update the disclaimer once the tariff is genuinely state-specific.

SAST-02 Required-field constraint declared but not enforced

MEDIUM

CWE	OWASP	DETECTED BY	FILE
CWE-1173 — Improper Use of Validation Framework	A04:2021 — Insecure Design	Manual review only	`custom-dropdown.tsx:100`

DESCRIPTION

`CustomDropdown` accepts a `required` prop and is used with `required` on the contact form and modal. The prop reaches the DOM as `aria-required` and a `data-required` attribute — neither of which the browser enforces. The backing `<input type="hidden">` that carries the value into the form **does not receive the real `required` attribute**:

```
// src/components/custom-dropdown.tsx:100
<input type="hidden" name={name} value={value}
       data-required={required || undefined} />   {/* ← not the `required` attribute */}

// :109 — advisory only; conveys intent to assistive tech, enforces nothing
<button ... aria-required={required || undefined} ...>
```

By contrast, the text fields in the same forms use the genuine attribute (`contact.tsx:26` , `contact-modal.tsx:55` — `required={!optional}`), so native constraint validation blocks submission when they are empty. The dropdown is the sole field exempt from that.

Note that `required` on a `type="hidden"` input is itself ignored by browsers per the HTML specification — so simply renaming the attribute would not fix this. The control needs explicit validation, as shown below.

IMPACT

The "Requirement type" field is presented to users with a red asterisk denoting a mandatory field, but the form submits without it. Currently the consequence is limited, because SAST-03 means no submission goes anywhere. The finding matters because it establishes a **false assumption of validation**: a developer reading this component reasonably concludes required fields are enforced. When the endpoint from SAST-03 is added, incomplete enquiries will reach it, and any server-side logic written on the assumption that `requirement` is always populated will be wrong.

REMEDIATION

Enforce the constraint explicitly in the component:

```
const [touched, setTouched] = useState(false);
const invalid = required && !value;

<input
  type="text"
  name={name}
  value={value}
  required={required}
  onChange={() => {}}
  aria-hidden
  tabIndex={-1}
  {/* visually hidden but still constraint-validated by the browser */}
  style={{ position:"absolute", opacity:0, width:1, height:1, pointerEvents:"none" }}
  onInvalid={() => setTouched(true)}
/>
```

A visually-hidden *text* input participates in native form validation where a hidden input does not, so submission is blocked and the browser's own validation message appears. Pair it with an inline error message driven by `invalid && touched` for a clear user-facing signal.

> **PRINCIPLE**
>
> Client-side validation is a usability feature, never a security control. Whatever is done here, the endpoint added under SAST-03 must independently validate every field on the server and reject anything malformed — it cannot trust that this dropdown was populated.

SAST-04 Numeric bounds not enforced; unbounded calculation input

LOW

CWE	OWASP	DETECTED BY	FILE
CWE-1284 — Improper Validation of Quantity	A03:2021 — Injection (validation)	Manual review only	`savings-calculator.tsx`, `lib/solar.ts`

DESCRIPTION

The consumption input declares `min={1} max={5000}` and the unit-cost input `min={0} max={100}`. HTML range attributes constrain the spinner controls and participate in form validation, but they do not constrain a typed or pasted value bound directly to React state. Both handlers store the raw string:

```
// src/components/sections/savings-calculator.tsx:134
<input type="number" min={1} max={5000} value={usage}
       onChange={(e) => setUsage(e.target.value)} />   {/* no clamp */}

// src/lib/solar.ts:24 — lower bound only; nothing caps the upper end
const monthlyUnits = Math.max(0, Number(usage) || 0);
const cost         = Math.max(0, Number(unitCost) || 0);
```

IMPACT

The library handles malformed input safely — negatives clamp to 0 and non-numeric text becomes 0 via `Number(x) || 0`, so there is no `NaN` propagation or crash. The gap is the *upper* bound. Entering 999,999,999 produces a recommended plant size of 83,33,333 kW and a lifetime saving of ₹28,79,99,99,97,120 — rendered with full confidence alongside the other results:

```
                                  (clamped — safe)
                                  (NaN||0 — safe)
```

This is a presentation-quality defect today, not a security one: the calculation is client-side, affects only the user's own screen, and cannot cause resource exhaustion on a static site. It is reported because the same values become an **API input** the moment SAST-03 is remediated, at which point missing upper bounds are a resource-consumption concern (OWASP API4:2023) rather than a cosmetic one.

REMEDIATION

Clamp in the domain layer, so every caller inherits the bound:

```
// src/lib/solar.ts
const MAX_MONTHLY_UNITS = 100_000;
const MAX_UNIT_COST     = 100;

const monthlyUnits = Math.min(MAX_MONTHLY_UNITS, Math.max(0, Number(usage)    || 0));
const cost         = Math.min(MAX_UNIT_COST,     Math.max(0, Number(unitCost) || 0));
```

Clamping in `lib/solar.ts` rather than in the component keeps the guarantee with the calculation, so a future server-side caller of `calculateSolar` is protected automatically. Add a visible out-of-range message so a clamped result is never presented as if it were the user's input.

SAST-05 Raw HTML sink for JSON-LD without `<` escaping

INFO

CWE	OWASP	EXPLOITABLE	CROSS-REF
CWE-79 — XSS (latent)	A03:2021 — Injection	No — static data only	VAPT-07

DESCRIPTION & IMPACT

`src/app/page.tsx:33` is the only raw-HTML sink in the codebase. Its input, `organizationSchema`, is a module-scope constant declared 15 lines above containing static company metadata — no user input, URL data or external source reaches it, so **it is not exploitable as written**.

The reason to record it: `JSON.stringify` does not escape `</script>`. The unsafe pattern is already in place and only the untrusted data source is absent. Should this schema ever be populated from a CMS or product feed, it becomes stored XSS with no further code change required.

REMEDIATION

```
const safeSchema = JSON.stringify(organizationSchema).replace(/</g, "\\u003c");
<script type="application/ld+json" dangerouslySetInnerHTML={{ __html: safeSchema }} />
```

Low priority. Apply when next editing the file, and treat any change making this data dynamic as security-relevant.

SAST-06 Build scripts excluded from type checking

INFO

CWE	OWASP	DETECTED BY	FILE
CWE-1109 — Inconsistent Enforcement	A05:2021 — Misconfiguration	Configuration review	`tsconfig.json`

DESCRIPTION

The project enables `strict: true` — commendable, and the source passes it cleanly with no escapes. However `scripts/` is explicitly excluded, so the two build scripts receive no type checking at all:

```
// tsconfig.json
"include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts", "**/*.mts"],
"exclude": ["node_modules", "scripts"]
```

IMPACT

Minimal today. Both scripts were reviewed manually and are sound: they contain hardcoded paths, perform no shell execution, accept no arguments, and read no environment variables (see 8.2). The concern is coverage discipline — `scripts/import-assets.ts` writes into `public/`, the directory published without access control, and it runs with developer privileges. Code that writes to the published output directory should be held to at least the same standard as code that renders it.

REMEDIATION

Remove `"scripts"` from `exclude`. If the scripts fail type checking under Bun-specific globals such as `Bun.write`, add `@types/bun` as a dev dependency rather than restoring the exclusion.

SAST-07 Non-null assertion bypasses strict null safety

INFO

CWE	REACHABLE	DETECTED BY	FILE
CWE-476 — NULL Pointer Dereference	No — data is static	Manual review only	`footer.tsx:158`

DESCRIPTION & IMPACT

A single non-null assertion appears in the codebase — the only place strict null checking is overridden:

```
// src/components/sections/footer.tsx:158
<Link href={linkColumns[0].headingHref!} className="hover:text-primary-300">
```

`headingHref` is optional on the link-column type but is defined for element 0 in the static data, so this cannot currently fail. If a future edit reorders `linkColumns` or removes that property, `href` becomes `undefined` and Next.js's `Link` throws at render time, breaking the footer on every page. The assertion converts a compile-time error into a runtime one.

REMEDIATION

```
{linkColumns[0].headingHref ? (
  <Link href={linkColumns[0].headingHref} className="hover:text-primary-300">
    {linkColumns[0].heading}
  </Link>
) : (
  linkColumns[0].heading
)}
```

Alternatively, make `headingHref` required on the type if every column is intended to have one — the better fix if the optionality was unintentional.

SAST-08 Unsound type assertions in hash validators

INFO

CWE	EXPLOITABLE	DETECTED BY	FILES
CWE-704 — Incorrect Type Conversion	No — guarded by Set.has()	Manual review only	2 detail components

DESCRIPTION & IMPACT

The hash validators analysed in Section 3.3 assert a `string` into a narrow union type in order to perform the `Set.has()` check:

```
// solar-module-detail.tsx:16 (identical shape at manufacturing-detail.tsx:18)
return moduleIds.has(hash as ModuleRange["id"])
  ? (hash as ModuleRange["id"])
  : null;
```

The runtime behaviour is correct and secure — this is a type-level observation only. The assertion is unsound in the sense that it tells the compiler something not yet proven, but the `Set.has()` guard makes it true in practice on the returning branch. The risk is that the pattern reads as "cast away the type error", which invites a future edit to keep the cast while dropping the guard — silently removing the application's only input validation.

REMEDIATION

Express the check as a type predicate so the compiler enforces the guard:

```
function isModuleId(value: string): value is ModuleRange["id"] {
  return (moduleIds as Set<string>).has(value);
}

function getModuleIdFromHash(): ModuleRange["id"] | null {
  const hash = window.location.hash.replace("#", "");
  return isModuleId(hash) ? hash : null;   // no assertion; narrowing is proven
}
```

This is a hardening refactor of already-correct code. Its value is that the guard becomes structurally load-bearing — removing it would then produce a compile error rather than a silent security regression.

SAST-09 IDE configuration committed; internal dev IP in config

INFO

CWE	OWASP	DETECTED BY	SCOPE
CWE-200 — Information Exposure	A05:2021 — Misconfiguration	Repository review	Repository hygiene

DESCRIPTION

Two minor exposures in version-controlled files:

1. **IDE configuration is tracked.** Five files under `.idea/` are committed, and the directory is not listed in `.gitignore`.
2. **A private network address is committed.** `next.config.ts` contains `allowedDevOrigins: ["192.168.18.157"]` — a developer's LAN address on the local network.

IMPACT

Both are low-value disclosures, and the assessment confirmed no sensitive content. All five `.idea/` files were read: they contain only `$PROJECT_DIR$` placeholder variables, module type declarations and an ESLint inspection toggle. There are no absolute filesystem paths, usernames, tokens or datasource definitions. JetBrains' own generated `.idea/.gitignore` already excludes the higher-risk files (`workspace.xml`, `dataSources.local.xml`), which is why nothing sensitive has been committed so far.

`192.168.18.157` is an RFC 1918 private address, not routable from the internet and of no direct use to an external attacker. It reveals only the internal subnet convention in use.

Both are reported as hygiene rather than exposure: the risk is that the current safety is incidental. A developer who adds a datasource, or a future tool that writes credentials into a tracked config file, would commit it without any control catching the mistake.

REMEDIATION

```
# Stop tracking IDE config and prevent recurrence
```

For the dev origin, read it from the environment so the address is not committed:

```
// next.config.ts
allowedDevOrigins: process.env.DEV_ORIGIN ? [process.env.DEV_ORIGIN] : [],
```

Consider adding a secret-scanning pre-commit hook (gitleaks or similar). The repository is clean today — verified across full history — and a hook keeps it that way as the team grows.

6 Dependency Position

Composition analysis is reported in full as VAPT-04 of the companion report and is summarised here for completeness, since dependency risk is conventionally part of a SAST engagement.

PACKAGE	INSTALLED	UPSTREAM	FIXED IN	REACHABLE IN PRODUCTION
next	16.2.10	HIGH	16.2.11	No — 9 advisories, all require a server runtime
postcss	8.4.31	HIGH	8.5.23	No — build-time CSS processing only
sharp	0.34.5	HIGH	0.35.0+	No — image optimizer disabled
nanoid	3.3.15	HIGH	3.3.16	No — transitive, build-time
brace-expansion	1.1.16	HIGH	1.1.17	No — transitive via ESLint, dev only

Production risk is negligible — every advisory requires a server-side execution context this static export does not have. Build-pipeline risk is real, since the PostCSS path-traversal advisories execute with the privileges of the CI runner. Remediation is a single patch upgrade:

```
$ npm update next@16.2.11 && npm update && npm audit && npm run build
```

7 Remediation Plan

#	ID	ACTION	PRIORITY	EFFORT	FILE
1	SAST-01	Wire `STATE_TARIFFS` into the calculation; resolve the dead category control	IMMEDIATE	2–3 hours	`savings-calculator.tsx`
2	SAST-03	Remove the unsupported 24-hour promise pending the endpoint	IMMEDIATE	< 1 hour	`contact.tsx`
3	SAST-03	Build the endpoint; derive success from the server response	2 WEEKS	2–3 days	3 components
4	SAST-02	Enforce the required constraint with a validated hidden text input	2 WEEKS	2 hours	`custom-dropdown.tsx`
5	SAST-04	Clamp numeric inputs in the domain layer	2 WEEKS	1 hour	`lib/solar.ts`
6	SAST-09	Untrack `.idea/` ; move the dev origin to an env var	1 MONTH	30 min	`.gitignore`
7	SAST-06	Include `scripts/` in type checking	1 MONTH	30 min	`tsconfig.json`
8	SAST-08	Replace hash assertions with type predicates	OPPORTUNISTIC	30 min	2 components
9	SAST-05	Escape `<` in the JSON-LD sink	OPPORTUNISTIC	10 min	`page.tsx`
10	SAST-07	Replace the non-null assertion with a conditional	OPPORTUNISTIC	10 min	`footer.tsx`

SEQUENCING

Items 4 and 5 should land *with or before* item 3. Both are validation controls whose absence is currently harmless only because no submission endpoint exists. Shipping the endpoint first means it receives incomplete and unbounded input from day one.

7.1 Recommended pipeline controls

The codebase is clean enough that continuous enforcement will preserve the position at low cost. All three engines already pass, so adding them to CI introduces no backlog:

CONTROL	RECOMMENDATION
Semgrep in CI	Run the same 7 rulesets on every pull request; fail on any new finding. Currently a zero-finding baseline.
ESLint + tsc gate	Both already pass — make them blocking so the clean state cannot regress.
Dependency scanning	Enable Dependabot or Renovate; add `bun audit` as a non-blocking visibility step.
Secret scanning	Add a gitleaks pre-commit hook. History is verified clean; this keeps it clean.
Unit tests for `lib/solar.ts`	Currently no test covers the calculation. SAST-01 and SAST-04 would both have been caught by a handful of assertions against known tariff/usage pairs.

HIGHEST-LEVERAGE SINGLE CONTROL

The last row is the most valuable item in this table. The two most consequential findings in this report — a 54% error in a consumer-facing financial projection, and unbounded calculation input — are both defects in one 47-line pure function with no I/O and no dependencies. It is the easiest thing in the codebase to test and currently the least protected. A short test file covering a few state/usage combinations would have caught both before release, and would prevent their reintroduction.

8 Appendix

8.1 Weakness coverage matrix

WEAKNESS CLASS	RESULT	NOTES
Code injection (CWE-94/95)	CLEAN	No dynamic evaluation construct present
Cross-site scripting (CWE-79)	INFO	SAST-05 — single sink, static input, not exploitable
DOM-based XSS (CWE-79)	CLEAN	Only taint path allow-listed — Section 3.3
Open redirect (CWE-601)	CLEAN	All 49 dynamic URLs from static data
SQL / command injection	N/A	No datastore, no shell execution
Path traversal (CWE-22)	CLEAN	No filesystem access from application code
SSRF (CWE-918)	N/A	No outbound requests
Deserialization (CWE-502)	N/A	No deserialization of untrusted data
Prototype pollution (CWE-1321)	CLEAN	No recursive merge or dynamic key assignment
Hardcoded credentials (CWE-798)	CLEAN	Semgrep `p/secrets`: 0 findings
Insecure randomness (CWE-338)	CLEAN	Reviewed — see 8.2
ReDoS (CWE-1333)	CLEAN	Reviewed — see 8.2
Improper input validation (CWE-20)	FINDINGS	SAST-02, SAST-04
Incorrect calculation (CWE-682)	FINDING	SAST-01
Control-flow defect (CWE-670)	FINDING	SAST-03
Null dereference (CWE-476)	INFO	SAST-07 — one assertion, currently unreachable
Type-confusion (CWE-704)	INFO	SAST-08 — guarded at runtime
Resource leak (CWE-401/772)	CLEAN	All listeners and timers released — Section 3.5
Information exposure (CWE-200)	INFO	SAST-09 — repository hygiene
Vulnerable components (CWE-1395)	FINDING	Section 6 / VAPT-04

8.2 Patterns reviewed and dismissed

Constructs that commonly produce SAST findings, examined here and determined **not** to be defects. Recorded so the same ground is not re-litigated at the next assessment.

PATTERN	LOCATION	WHY IT IS NOT A FINDING
Custom LCG pseudo-random generator	`generate-sunburst.ts:7`	Deterministic by design — seeded so regenerating the decorative SVG produces an identical file in git. Output positions squares in a graphic; it is never used for tokens, IDs or any security purpose. Replacing it with a CSPRNG would break reproducibility for no benefit.
Regular expressions on user-adjacent strings	`count-up.tsx:27`	`/[\d,]*\d/g` has no nested quantifier and cannot backtrack catastrophically. Its input is a static prop, not user data.
Image processing on untrusted files	`import-assets.ts:32`	`sharp` processes a fixed list of four hardcoded design-export paths at build time. No user-supplied file reaches it.
Module-scope mutable singleton	`reveal.tsx:14`	A shared `IntersectionObserver` with a `WeakMap` callback registry — an intentional performance optimisation for ~40 observed elements. Entries are removed on unmount and the WeakMap permits collection.
Modulo on array length	`custom-dropdown.tsx:76`	Would produce `NaN` if `options` were empty, but every call site passes a non-empty static array. Not reachable.
Direct `document` access in effects	5 components	All reads (`getElementById` , `querySelector`) for scroll positioning, guarded by null checks. No untrusted value reaches a selector.

8.3 Reproduction

```
# Semgrep — 129 rules across 7 security rulesets
# [illegible]
# [illegible]
# [illegible]
# [illegible]

# ESLint — flat config, Next.js core-web-vitals + TypeScript
# [illegible]

# TypeScript — strict mode, no emit
# [illegible]

# Dependency advisories
# [illegible]
```

8.4 Limitations

- **Point-in-time.** Findings reflect commit `a4f4020` as of 17 August 2026.

- **First-party code only.** `node_modules/` was not statically analysed; third-party risk was assessed by composition analysis against advisory databases (Section 6).
- **No dynamic execution.** This was a static analysis. Runtime behaviour of the deployed site is covered by the companion VAPT report.
- **Generated output excluded.** `.next/` and `out/` are build artifacts derived from the reviewed source and were not separately analysed.
- **Automated tool limits.** A zero-finding result from the three engines means no recognised dangerous pattern was matched. It is not evidence of correctness, which is why every finding in this report originated in manual review.

END OF REPORT

ReNew Solar Marketing Website — SAST Report v1.0 — 17 August 2026. 6,689 lines across 41 files analysed. 9 findings: 0 Critical, 1 High, 2 Medium, 1 Low, 5 Informational. Automated engines: 0 findings. All findings from manual review.